

Classifying and Predicting NBA Injuries (2021-2024)

Home assignment - Written Product

Student Number: 183709

Exchange - Undergraduate

17 December 2025

Supervisor: Professor Coen Van der Geest

23,254 Characters with spaces, 13 Pages (15 with References)

Intro

In the National Basketball Association (NBA), player availability is often the deciding factor between winning a championship and missing the playoffs. Injuries not only affect the immediate outcome of games but also have a long-term influence on finances, trading power, and strategy for franchises. The league has attempted to curb player absences through the Player Participation Policy (PPP) and 65-game rule for the 2023-2024 season, which requires a minimum of 65 games played by players to be eligible for awards and bonuses (Aldridge, 2025). However, the reality is far more complicated than players sitting out for “load management” or “rest”. During the most recent NBA Season, there has been a sharp decline in star players, such as the Celtics’ forward, Jayson Tatum, and the Pacers’ Tyrese Haliburton, both top 10 players, being sidelined due to injuries during the previous season.



Note. Image is sourced from (Press, 2025)

The league is facing a wave of injuries, a lot involving tendons that have recovery times of 6-12 months. The policy of 65 minimum games might, in fact, be counterintuitive in its purpose of increasing the number of stars playing. More games usually mean more wear and tear on the body, less time for the body to recover, and thus an increased risk of injury.

It does not help that the game is changing towards a more physical style. According to The Athletic, the “NBA pace is the fastest it’s ever been since the 1991-92 season, players are running faster and further than ever before” (Aldridge, 2025).

The goal of this exploratory data analysis and modeling is to see if we can classify the severity and predict the length of injuries among NBA players based on medical factors, player statistics, and

seasonal timing. By applying Big Data Analytics to injury reports and player statistics, we might be able to move towards an approach that allows staff to plan injuries. This could extend into predicting which players are susceptible to specific injuries during certain points in the season.

Data Preparation

There was no official NBA dataset on injuries that was readily available for the public. The injury data was sourced from Vaughn Hajra (2024), who created an NBA injury list for the 2021-2024 seasons using pregame injury reports. Before every game, player status is reported as available, questionable, or out. The data source contained the report date, injury type, status, and player name. The data was formatted in a granular, day-by-day account of player status, which included specific reasons for absence (ex. "L. Knee Soreness") and "Status" (ex. "Out," "Questionable"). This level of data was essential for distinguishing between minor day-to-day soreness and a significant injury/event. However, injury data alone was insufficient for predictive modeling, so additional data had to be pulled from Sports Reference LLC (n.d.), an open-source sports database that contains season stats for all NBA players for each season. This additional source includes variables such as total season minutes, position, age, and team, which serve as proxies for “workload” and “playstyle,” allowing the model to test if specific positions or high usage players are more susceptible to certain types of injuries.

The raw injury data were unstructured, requiring transformation and cleaning to ensure accuracy and consistency. A major challenge in merging the datasets was the inconsistent naming conventions. Certain names of international players used special characters or accents, which were not found in the *Sports Reference* summary data. To resolve this, the code uses a Python library and function to normalize names by removing periods from initials, replacing unorthodox characters, and handling special cases for international player names, so I could merge the datasets. Next was removing the redundant reports that did not capture the goal of the model. The status “Questionable” or “Doubtful”, where players ultimately played in the game, was removed. The dataset was filtered to only rows where the Status was explicitly listed as “Out”. Additionally, redundant reasons for absence, such as “Load Management” or “Personal Reasons,” were filtered out, leaving only injury or illness as reasons for being absent.

Feature engineering was necessary to capture patterns that were not present in the raw data or text descriptions. I created a season variable based on the NBA calendar, where the months from October to December were assigned to the season ending in the following year. To test the hypothesis that physical wear and tear accumulates as the season progresses, I created a Season Phase variable which segments the season into “Early” (Oct to Nov), “Mid” (Dec to Feb), and “Late” (March – April) categories. Also, the “Reason” field in the raw data was messy and contained lots of unique descriptions. I used a Python library, Regular Expressions, to filter these strings and create a cleaned version of the injury (W3Schools, 2019). These cleaned injury descriptions were then used to create a

new variable, Body Part Groups, to reduce high dimensionality and improve model performance (Van der Geest, 2025b).

The problem with total season statistics for each player is that they can leak if a player gets injured. For example, if a player has a high number of total points, then they likely were healthy all season, or if they had a high shooting volume. To mitigate this, I calculated the averages based on games played (ex. average minutes, points, free throw attempts, and 3-point attempts). While this method assumes that player averages remain relatively stable across a season, it provides a robust estimate of a player's typical performance without leaking the total duration of their absence. and thus, gives us an estimate of what their previous career stats were like. This version of the injury dataset was merged with the stats to create a Tableau visualization-compatible dataset.

A critical issue with the current dataset is that a player missing 10 consecutive games would appear as 10 separate rows, leading to data leakage and overcounting of injuries if used directly in modeling. It is important to remember that the goal is to predict the time out from each injury incident, not every report of the same recurring injury. To solve this, I used a function that collapsed sequential reports into a single injury report according to specific rules. If the time gap between reports for the same player and body part was greater than ten days, or if it was the first record for that player, it was flagged as a new injury event. I then grouped the rows by these unique identifiers to create a Duration Days variable, representing the true length of each injury.

Another feature that was added to help predictive modeling was a list of the players rank the previous season and the number of injuries they had that season. After all, the goal is to predict the severity of injuries, rank provides a strong proxy for usage besides average minutes per game, and the number of injuries the previous season could be a strong indicator of whether a player is likely to have severe injuries this season.

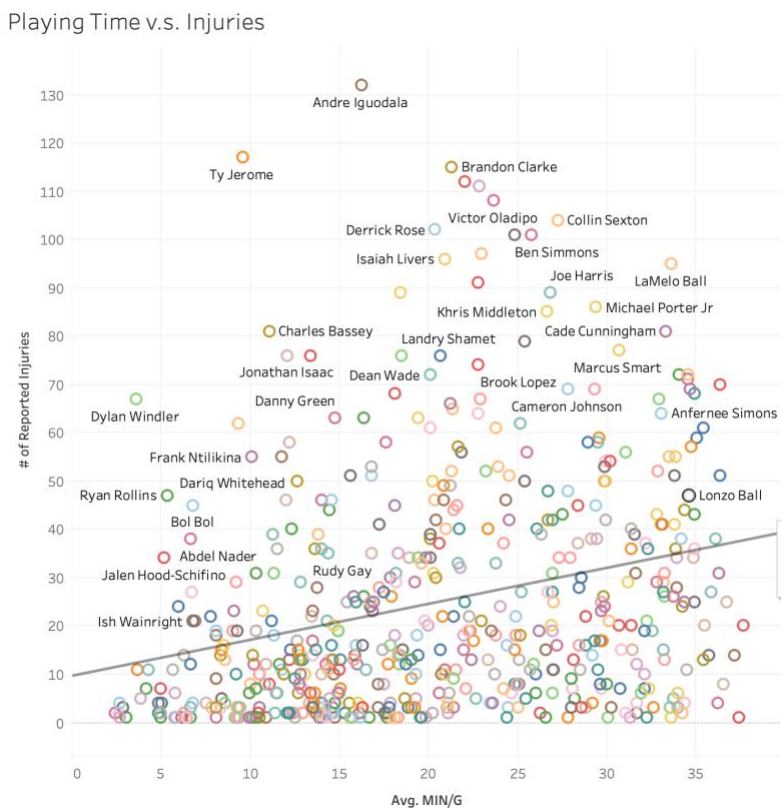
For the unsupervised model, the data was aggregated down so that there was only one row for every injured player each season, containing their season stats, total injuries that season, as well as general information.

Finally, this collapsed injury dataset was merged with the seasonal performance statistics using a left join on player name and season. To avoid duplication, I filtered the data to only use the TM "Total" row for traded players, which ensured that their season-long stats were represented. This cleaning and processing resulted in two distinct datasets. There is a visualization dataset that contains daily frequency to show injury trends over time, and a machine learning dataset containing unique, independent injury events, both merged with season performance and stats.

Analysis

Before applying and creating models for prediction, an exploratory analysis is conducted to gain a deeper understanding of the data's underlying structure and identify trends. The goal was to gain a general understanding of the list of players, their injury histories, and overall usage.

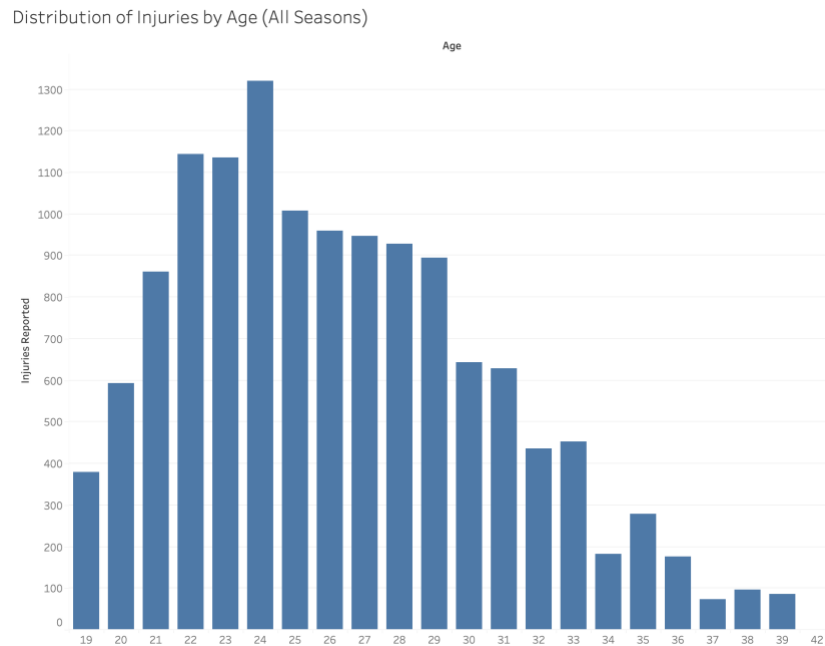
Figure 1: Relationship between Playing Time (AVG Mins per Game) and Injury Frequency



Note. Data from *Creating an NBA injury database* (Hajra, 2024) and *NBA player stats* (Sports Reference LLC, n.d.)

On this plot, some popular players appear, notorious for their injuries, such as LaMelo Ball, Derrick Rose, and Victor Oladipo. There is a clear pattern: higher playing time correlates with more injuries throughout the season.

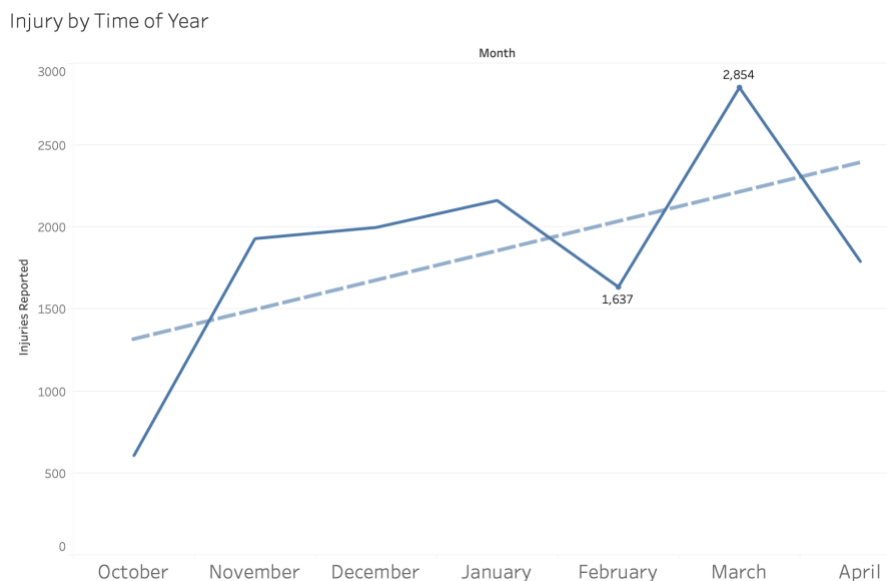
Figure 2. Age Distribution of Injured NBA Players, demonstrating a bell-shaped curve



Note. Data from *Creating an NBA injury database* (Hajra, 2024) and *NBA player stats* (Sports Reference LLC, n.d.)

Visualizing the distribution of injuries by age reveals an almost normal-like distribution, centered around age 24, which aligns with the average age of NBA players. The number of injuries drop significantly after age 30. This could be due to survivorship bias, as more injury-prone players tend to retire earlier, such as Derrick Rose.

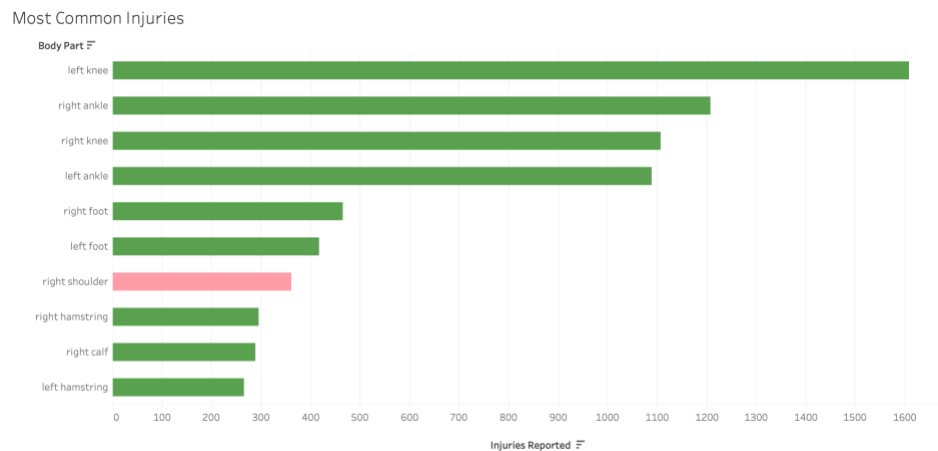
Figure 3. *Line chart of Injuries by Month, showing an upward trend throughout the season*



Note. Data from Creating an NBA injury database (Hajra, 2024) and NBA player stats (Sports Reference LLC, n.d.)

While injury volume remains relatively constant in the early season, we start seeing a jump in injuries during the last third of the season, peaking in March. It is important to note the drop in injuries during February, as this is the time when the All-Star game takes place, and players get a week of rest. This is not indicative that this one week of rest reduced injuries, but rather that injuries were not reported during this one week since there were no games. This is a similar case in April, as the start of the playoffs coincides with a rapid decline in the number of teams playing, resulting in a corresponding decrease in reported injuries. However, the general trend upwards, especially in March, indicates the wear and tear effect is real, as the season goes on, more injuries get reported.

Figure 4. *A deeper analysis of the most common injuries, color-coded by body group*



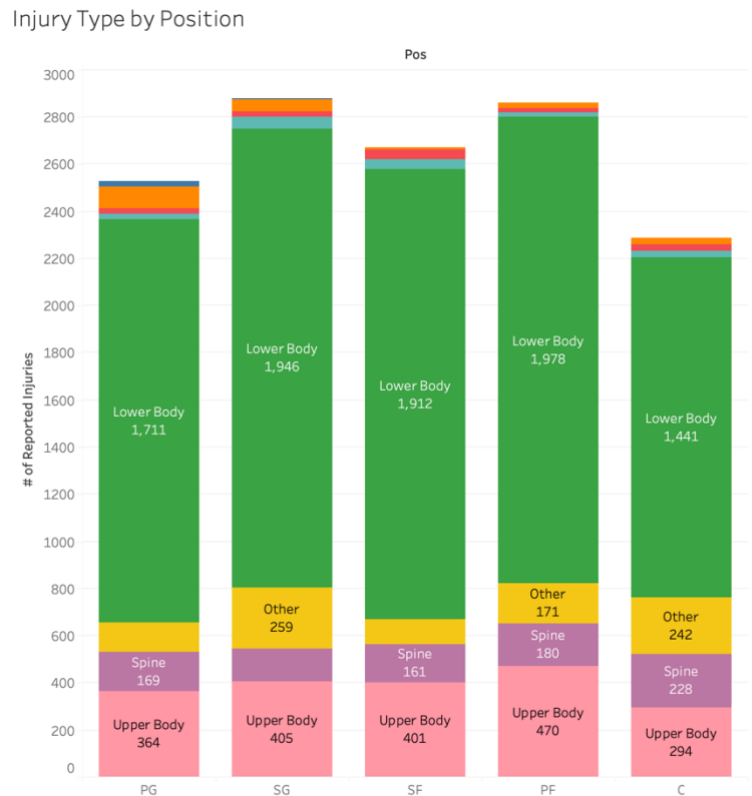
Note. Data from Creating an NBA injury database (Hajra, 2024) and NBA player stats (Sports Reference LLC, n.d.)

The knees and ankles are the leading causes of injuries among basketball players. However, there was a noticeable difference between left and right knee injuries. This could be evidence of the dominant leg effect, where players are more likely to injure their dominant limb due to muscle imbalance and biomechanics. A study done by Matthew DeLang from Texas Health Sports Medicine (2021) found that soccer players have a 1.6 times higher risk of injury to the dominant limb. Although this study was done on soccer players, the similarities can be carried over to basketball.

Next was exploring and dissecting injuries based on player position, a general indicator of playstyle, to see if there were any patterns. In basketball, guards (Point Guard, Shooting Guard) tend to be more mobile players who tend to run the most out of all positions, involving rapid changes of direction and handling the ball. Forwards (Small Forward, Power Forward) are more of slashers, who

deal with more physical contact from other players and cutting to the basket. This position often involves rebounding, so lots of jumping near the rim. Centers are the tallest and heaviest players on the court whose responsibility consists of rebounding and rim protection, often placing a lot of stress on their joints and back with all the physical contact they receive.

Figure 5. *Stacked Bar Chart analyzing injury types and groups by player position*

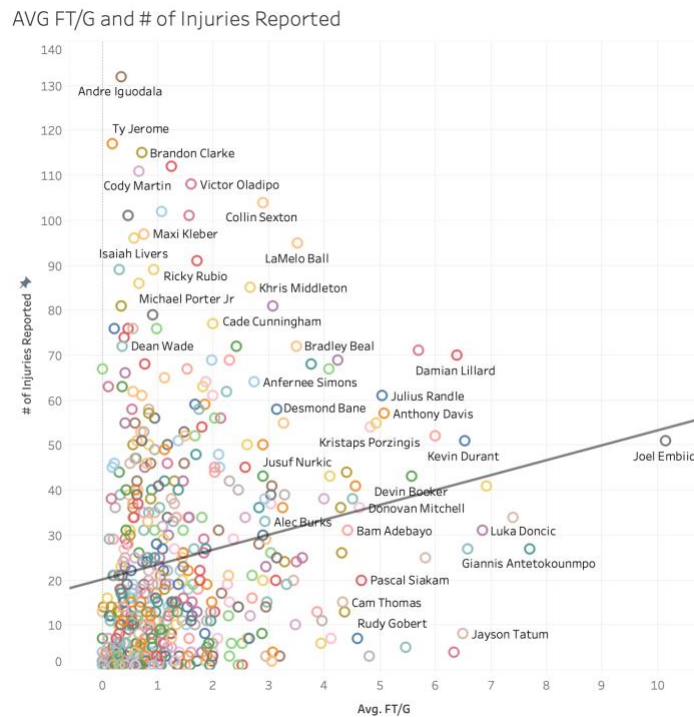


Note. Data from *Creating an NBA injury database* (Hajra, 2024) and *NBA player stats* (Sports Reference LLC, n.d.)

The data shows the positions with the most injuries were the shooting guard and power forward positions. Centers had a noticeably lower number of injuries, which could be due to centers playing on average fewer minutes than other positions or not requiring as much cutting and agility. However, centers do have more spine injuries than any other position, possibly attributed to the physical play style and constant jumping.

When thinking about NBA stats that could explain injuries, free throws come to mind.

Figure 6. *Scatter plot of the average Free Throws per game correlating with injuries*



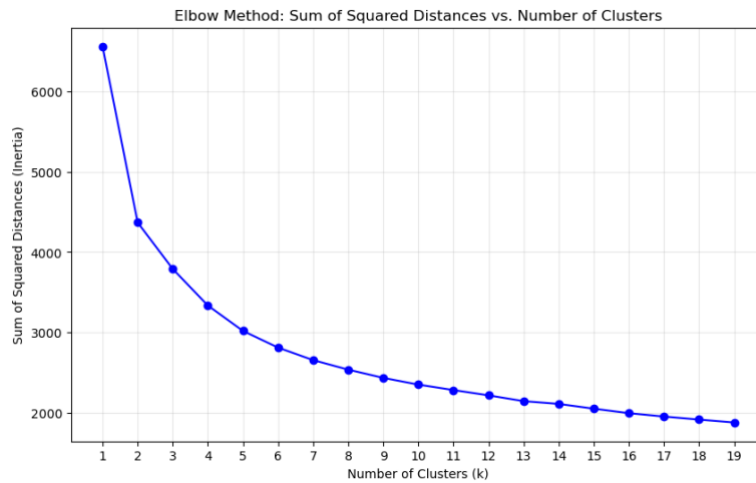
Note. Data from *Creating an NBA injury database* (Hajra, 2024) and *NBA player stats* (Sports Reference LLC, n.d.)

The results strongly supported that more free throws per game correlate strongly with more reported injuries for a player. In fact, the slope of the trend line graphed to this scatter plot was significant with a p -value < 0.001 and a slope of around 3.3 (see Appendix A for full regression statistics). This means if a player's average free throws per game goes up by 1, then they, on average, will have 3.3 more injury incidents reported that season. Teams can plan and predict injuries with this new information, identifying high-risk free-throw players and planning ways to prevent their injuries.

The next step was to create a model that could identify hidden patterns in the data. To complete this, I used K-Means Clustering, an unsupervised learning technique. The goal was to discover if there were certain "risk profiles" among players who were injured based on their workload and injury history (Van der Geest, 2025c).

A comprehensive set of features, such as Age, Average Minutes Per Game (MPG), Average Points, Free Throws, and the cumulative count of injuries during a season, were selected. The Elbow Method was used to find the optimal number of clusters by calculating the Sum of Squared Distances for different numbers of clusters (Van der Geest, 2025c).

Figure 7. Line graph of the Sum of Squared Distances versus Number of Clusters



Note. Model created using scikitlearn library (Pedregosa et al., 2011) (refer to Appendix D for code)

Based on the chart above, a cluster of $k=4$ provided the most distinct separation of player types without becoming too specific.

The next step was the supervised learning model, where two different models were used to see if one performed better at classifying player injuries. The Severity label was based on the number of games missed from each injury event, where “Low” was for less than 3 games missed, “Medium” for 3 to 7 games missed, and “High” was for anything greater. The logistic regression model was set to multinomial, as it must predict 3 separate classes, not just a binary one (Pedregosa et al., 2011). The Random Forest was selected as a robust model as it is prone to overfitting, can capture more complex relationships, and can handle class imbalances in the injury data (Van der Geest, 2025d). To prepare the data for supervised machine learning, a train-test split of 75/25 was implemented with a fixed seed to make sure the results were reproducible (Pedregosa et al., 2011). The Precision and Recall were prioritized for the “High Severity” class, as in a real-world context, accurately identifying long-term injuries is the most informative and actionable insight for teams. Accuracy alone can be misleading due to the high volume of minor and insignificant injuries (Van der Geest, 2025d).

Results

To identify the distinct player profiles, a K-Means clustering was applied to the dataset using an optimal parameter of 4 clusters found using the Elbow Method (Van der Geest, 2025c). The algorithm successfully split the injured player dataset into 4 unique archetypes based on workload, age, basic performance, and injury frequency.

Table 1. *Table of 4 different cluster averages and their labeled archetype*

<i>Cluster</i>	<i>Age</i>	<i>MPG</i>	<i>Injuries During Season</i>	<i>AVG Points</i>	<i>AVG Free Throws</i>	<i>AVG 3 Pointers</i>	<i>Possible Archetype</i>
0	24.60	28.29	1.13	13.21	1.88	1.59	Durable Rotation
1	27.34	34.25	2.45	23.72	4.68	2.26	Elite Stars
2	23.25	14.05	0.79	5.10	0.70	0.53	Developing Rookies/Bench
3	31.34	19.77	1.95	7.25	0.94	0.90	Fragile Veterans

Note. Data from *Creating an NBA injury database* (Hajra, 2024) and *NBA player stats* (Sports Reference LLC, n.d.) (refer to Appendix D for code)

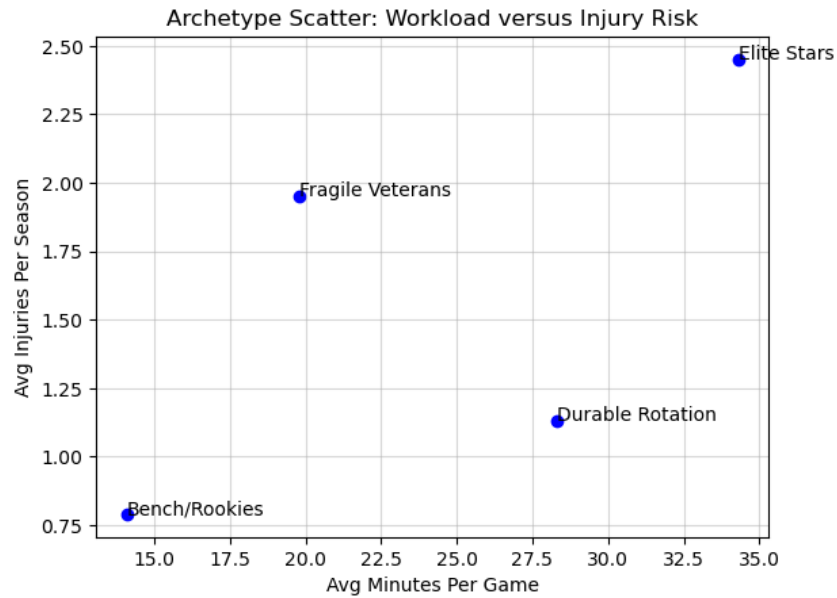
Cluster 0 is players who serve as rotational pieces. With an average age of 24.6 and playing around 28.3 out of the 48 minutes in a game, they clearly serve a key role, but are not the center of a team. They can be thought of as role players who are only put in during certain situations and can be pulled from a game if needed. Unlike stars, who are pressured because the entire team runs around them, role players have only a small responsibility, thus the lower average injuries.

Cluster 1 consists of the elite stars. These are big-name players, such as Luka Doncic, Tyrese Maxey, Giannis Antetokounmpo, who are the center of their team. They are in their physical prime age-wise (27.34) but also bear the most physical burden through high minutes (34.25) and scoring (23.72). This elite status comes with the highest cost, as this archetype sustains the most injuries by far, averaging 2.45 injuries per season. This confirms the major hypothesis that there is a trade-off between output and health risk, as the most valuable players are the most likely to miss time.

Cluster 2 is the bench and development players, those typically early into their NBA career, finding their role. They have the youngest average age of 23.3 and the lowest workload with only 14.1 MPG. Their low output also seems to coincide with the lowest injury frequency, with only 0.79 injuries per season. These players are generally protected from the physical toll of the full season, as they do not play every game and are at extremely low risk of injury.

The final Cluster, 3, consists of the high-risk veterans. This group has the highest average age of 31.3, but despite their lower minutes per game (19.8), they still have a disproportionately high number of injuries (1.95 per season). The numbers support my theory of a survivorship bias and wear and tear, as older players are more susceptible to injury even with a lower workload.

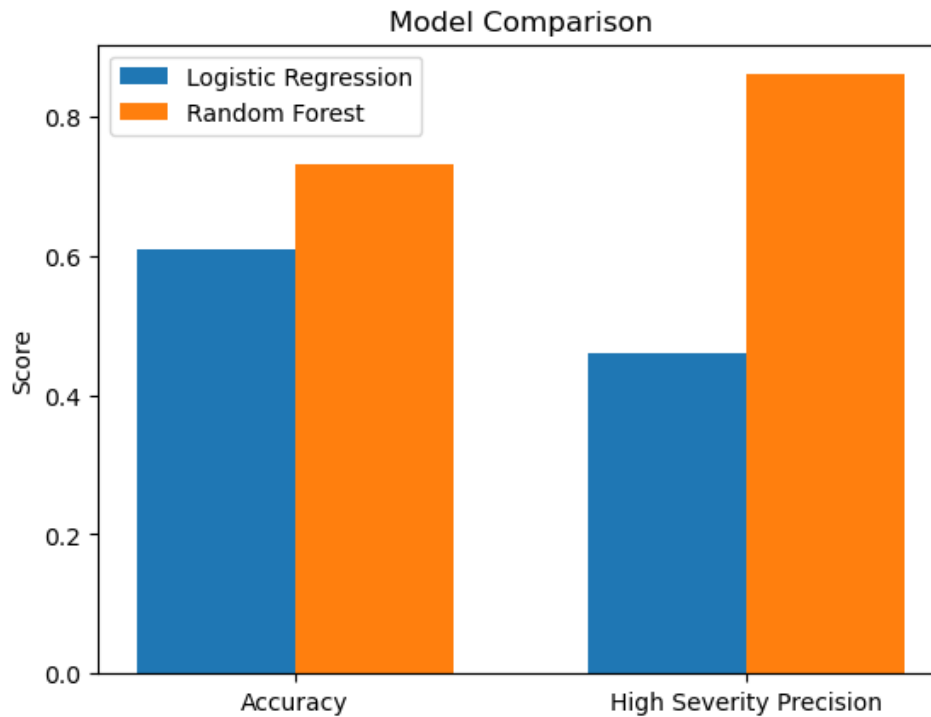
Figure 8. *Plot of Workload and Injuries for the different clusters*



Note. Data from *Creating an NBA injury database* (Hajra, 2024) and *NBA player stats* (Sports Reference LLC, n.d.) (refer to Appendix D for code)

In the classification task, the goal was to predict the severity of an injury (Low, Medium, or High) based on player workload and history. I compared a basic Logistic Regression model against a Random Forest Classifier. The following results were produced using the scikit-learn classification report function (Pedregosa et al., 2011). The Logistic Regression achieved an overall accuracy of 61.2%. However, diving into the confusion matrix shows that the Logistic Regression achieved this score by over-predicting the majority class, Low Severity. It had very low performance when identifying the more crucial business-impacting “High Severity” injuries, with a precision of only 46%. In a real NBA team, this model would provide no valuable insight, as it is correct less than half the time when flagging a high-severity injury.

Figure 9. Bar chart comparing supervised model performance



Note. Data from *Creating an NBA injury database* (Hajra, 2024) and *NBA player stats* (Sports Reference LLC, n.d.) (refer to Appendix D for code)

On the other hand, the Random Forest Classifier achieved a significantly higher accuracy of 73.4%, greatly surpassing the logistic baseline. However, to understand its practical use, it is important to consider other metrics like precision and recall, especially for the critical “High Severity” class (Van der Geest, 2025d). The model reached a precision of 86% for high-severity cases, indicating that it is very reliable when identifying major injuries. For example, in an NBA situation, if the model predicts that a star player like Jayson Tatum will miss 8+ games, it is very likely to be correct. In terms of recall, the model only achieved 37%, which shows a high rate of false negatives. There will be cases where a player sustains a severe injury, but the model incorrectly predicts a short absence. These errors can have serious medical and operational consequences (Van der Geest, 2025d). Due to its low recall, it cannot be relied on to detect every serious injury, but its high precision means that when it does identify a serious injury, team managers should take notice.

Discussion

The business value of this lies in its ability to provide medical estimate timelines before the player even reaches the hospital. The Random Forest model demonstrated reliability with an 86% precision when flagging severe injuries. The false positive rate is so low that it allows management to

take expensive but decisive actions quickly, such as signing a replacement player or executing a trade. In the sports world, where games can be played the next day or star players switch teams overnight, this speed can make the difference between squeezing into the playoffs or missing them entirely.

This model can help teams bypass the typical uncertainty period after an injury, the 24–48-hour window where traditional medical evaluations are still ongoing. By utilizing the model’s high-confidence prediction of severe injuries, a manager can quickly switch a team’s strategy or plan for future games without waiting for a detailed evaluation from medical staff. Teams should use this model after a player gets injured during a game, and if the model predicts an injury as severe, take the appropriate action of replacing the player through trades or roster movements.

The unsupervised clustering analysis offers something that the supervised classification model does not, and that is proactive risk management. Being able to identify a player archetype will allow the training staff to allocate resources to support their performance and longevity. For cluster 3, veterans, they should focus on load management and preservation. Since these players have a high injury rate despite low minutes, they should be used strategically in key games against rivals. For cluster 1, stars, staff should focus on their recovery and lower-body health. These players cannot be rested due to their importance, so the goal should be to maximize their recovery to keep them in as many games as possible.

While the model can provide a competitive advantage, there are limitations. The model is conservative and misses a significant portion of severe injuries with its lower recall. It is important to note that this model is meant to estimate injury duration now. Ultimately, teams should rely on full medical assessments and data for a more precise diagnosis. There are ethical dilemmas to be aware of with this model, such as contract negotiations and player treatment. While this project uses public data, NBA teams have access to far more private health data, and there is a risk that such data, when used in a model, could be unfair to players. Teams could unfairly devalue players during contract negotiations due to the model “predicting” their injury risk. Management must make sure that these tools are used to protect players’ health rather than exploit their value.

References

- Aldridge, D. (2025, November 25). Many NBA stars are getting injured. Will the league and its partners dive deep for answers? *The New York Times*.
<https://www.nytimes.com/athletic/6833570/2025/11/25/nba-stars-injury-answers-schedule-length/>
- DeLang, M. D., Salamh, P. A., Farooq, A., Tabben, M., Whiteley, R., van Dyk, N., & Chamari, K. (2021). The dominant leg is more likely to get injured in soccer players: systematic review and meta-analysis. *Biology of Sport*, 38(3), 397–435. <https://doi.org/10.5114/biolsport.2021.100265>
- Hajra, V. (2024, November 17). *Creating an NBA Injury Database*. Substack.com; Stat Surge.
<https://statsurge.substack.com/p/creating-an-nba-injury-database>
- Sports Reference LLC. (n.d.). *2021-22 NBA player stats: Totals*. Basketball-Reference.com. Retrieved December 1, 2025, from https://www.basketball-reference.com/leagues/NBA_2022_totals.html
- Sports Reference LLC. (n.d.). *2022-23 NBA player stats: Totals*. Basketball-Reference.com. Retrieved December 1, 2025, from https://www.basketball-reference.com/leagues/NBA_2023_totals.html
- Sports Reference LLC. (n.d.). *2023-24 NBA player stats: Totals*. Basketball-Reference.com. Retrieved December 1, 2025, from https://www.basketball-reference.com/leagues/NBA_2024_totals.html
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Press, T. A. (2025, May 13). *Jayson Tatum injury: Celtics star hurt in loss to Knicks*. NBC News. <https://www.nbcnews.com/sports/nba/boston-celtics-jayson-tatum-seen-wheelchair-suffering-leg-injury-rcna206418>

- Van der Geest, C. (2025a). *Big Data and Analytics [Lecture Slides]*. Big Data Analytics for Managers, Copenhagen Business School.
- Van der Geest, C. (2025b). *Data Preparation [Lecture Slides]*. Big Data Analytics for Managers, Copenhagen
- Van der Geest, C. (2025c). *Intro to Clustering [Lecture Slides]*. Big Data Analytics for Managers, Copenhagen
- Van der Geest, C. (2025d). *Overfitting [Lecture Slides]*. Big Data Analytics for Managers, Copenhagen Business School.
- Van der Geest, C. (2025e). *Statistical Models [Lecture Slides]*. Big Data Analytics for Managers, Copenhagen Business School.
- W3Schools. (2019). *Python RegEx*. W3schools.com
https://www.w3schools.com/python/python_regex.asp

Appendix

Appendix A: Tableau Trendline Output (Free Throws per Game v.s. Injuries)

Model formula: (Avg. FT/G + intercept)
Number of modeled observations: 532
Number of filtered observations: 0
Model degrees of freedom: 2
Residual degrees of freedom (DF): 530
SSE (sum squared error): 308713
MSE (mean squared error): 582.477
R-Squared: 0.036033
Standard error: 24.1346
p-value (significance): < 0.0001

Individual trend lines:

Panels		Line		Coefficients				
<u>Row</u>	<u>Column</u>	<u>p-value</u>	<u>DF</u>	<u>Term</u>	<u>Value</u>	<u>StdErr</u>	<u>t-value</u>	<u>p-value</u>
Count of Reason	FT/G	< 0.0001	530	Avg. FT/G	3.31425	0.744608	4.451	< 0.0001
				intercept	20.1131	1.49301	13.4715	< 0.0001

Appendix B: Python Supervised Model Output (Logistic Regression and Random Forest Models)

```

Logistic Regression Model Results
Accuracy: 0.6118881118881119
[[ 55 150  16]
 [ 41 586  42]
 [ 23 172  59]]

```

	precision	recall	f1-score	support
High	0.46	0.25	0.32	221
Low	0.65	0.88	0.74	669
Medium	0.50	0.23	0.32	254
accuracy			0.61	1144
macro avg	0.54	0.45	0.46	1144
weighted avg	0.58	0.61	0.57	1144

```

Random Forest Model Results
/Applications/anaconda3/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1247: F
warnings.warn(
Accuracy: 0.7342657342657343
[[ 81 132   8]
 [ 10 654   5]
 [   3 146 105]]

```

	precision	recall	f1-score	support
High	0.86	0.37	0.51	221
Low	0.70	0.98	0.82	669
Medium	0.89	0.41	0.56	254
accuracy			0.73	1144
macro avg	0.82	0.59	0.63	1144
weighted avg	0.77	0.73	0.70	1144

Appendix C: Python Unsupervised Model Output (K-Means Clustering Model)

Size of each cluster

Cluster

```
0    281
1    139
2    313
3    232
```

Name: count, dtype: int64

Cluster Average Numeric Variables

	Age	MPG	InjuriesThisSeason	AVGPTS	AVGFTS	\
Cluster						
0	24.604982	28.287800	1.128114	13.212348	1.882658	
1	27.345324	34.254389	2.446043	23.720711	4.675742	
2	23.258786	14.050900	0.785942	5.098690	0.704250	
3	31.340517	19.776229	1.952586	7.249860	0.938662	

AVG3PTS

Cluster

```
0    1.589043
1    2.256318
2    0.531955
3    0.897495
```

...

```
0    0.131673  0.156584  0.163701  0.256228  0.291815
1    0.100719  0.237410  0.323741  0.129496  0.208633
2    0.201278  0.214058  0.130990  0.198083  0.255591
3    0.228448  0.250000  0.198276  0.172414  0.150862
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

Appendix D: Full Python Code

```
# %%
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.linear_model import LogisticRegression
import re
import unicodedata
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# %%

#Loading and preparing injury DF
injury_df = pd.read_csv("Injury2021-2024.csv")

# %% [markdown]
# Injury Dataset Transformations
# The injury dataset contains the injury/status report of players before every game. In order to identify each injury and not
# over count, I will identify when a new injury begins and keep track of how many games they miss
#
# Two separate datasets are being created for a clustering model (unsupervised learning) classification model (supervised)
# and Data visualizations
#
# General Transformations
# - Normalize player names
# - Create season variable
# - Drop non-injury rows
# - Creating injury location and type variable
# - Season phase variable for time of injury
# - Count of prior injuries per player/season
#
# Dataset A (Injury Event, ML models)
# - Collapsed down to each injury event
#
# Dataset B (Raw Injury Report, For Visualizations)
#

# %%

#Injury Dataset Transformation
#Standardizing Name

```

```

def fix_name(name):
    if pd.isna(name):
        return name
    if "," in name:
        last, first = name.split(",", 1)
        return first.strip() + " " + last.strip()
    else:
        return name.strip()

injury_df['PLAYER'] = injury_df['PLAYER'].apply(fix_name)

def normalize_initials(name):
    if pd.isna(name):
        return name

    # normalize names
    name = re.sub(r"\b([A-Za-z])\.\s*([A-Za-z])\.\b", r"\1\2", name)
    name = re.sub(r"\b([A-Za-z])\.\b", r"\1", name)
    name = name.replace(".", "")

    return name.strip()

injury_df["PLAYER"] = injury_df["PLAYER"].apply(normalize_initials)

#Creating Season Variable
injury_df['DATE'] = pd.to_datetime(injury_df['DATE'])
injury_df['Year'] = injury_df['DATE'].dt.year
injury_df['Month'] = injury_df['DATE'].dt.month
injury_df['Day'] = injury_df['DATE'].dt.day

#October of year 0 to June of year 1 is full season
#Season 2022-2024
#If month is Oct, Nov, Dec, then the season is the following year, else it is the current year

```

```

injury_df['Season'] = np.where(injury_df['Month'] >= 10, injury_df['Year'] + 1, injury_df['Year'])

#Dropping non-injury Rows and keeping only injuries that resulted in player being "Out"
injury_df = injury_df[injury_df['REASON'].str.contains("Injury/Illness", case = False, na = False)]
injury_df = injury_df[injury_df['STATUS'].str.contains("Out", case = False, na = False)]

#Creating body part and injury detail variable
# Body Part
injury_df['BodyPart'] = injury_df['REASON'].str.extract(r'\s*(.*?)', expand=False)

#Creating grouping for body part, its really messy
injury_df["BodyPart_clean"] = (
    injury_df["BodyPart"]
    .astype(str)
    .str.lower()
    .str.replace(r"^[a-z0-9]\s", "", regex=True)
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)

def map_body_part_group(x):
    if pd.isna(x) or x in ["", "na", "n/a", "not available", "", " ", "-"]:
        return "Other"

    # LOWER BODY
    if any(k in x for k in [
        "ankle", "foot", "toe", "knee", "hamstring", "quad", "thigh", "calf", "shin",
        "achilles", "hip", "groin", "adductor", "tibia", "fibula", "leg", "midfoot"
    ]):
        return "Lower Body"

    # UPPER BODY
    if any(k in x for k in [

```

```

    "shoulder", "elbow", "wrist", "hand", "finger", "thumb", "forearm",
    "arm", "bicep", "tricep", "metacarpal", "radius", "ulna"
}):
    return "Upper Body"

# SPINE
if any(k in x for k in [
    "back", "lumbar", "spine", "cervical", "disc", "l5", "l4", "mid back", "low back"
]):
    return "Spine"

# CORE
if any(k in x for k in [
    "abdomen", "abdominal", "core", "oblique", "rib", "stomach"
]):
    return "Core"

# HEAD / FACE
if any(k in x for k in [
    "concussion", "head", "face", "facial", "eye", "jaw", "nose", "orbital", "dental"
]):
    return "Head/Face"

# RESPIRATORY / CHEST
if any(k in x for k in [
    "chest", "sternum", "lung", "pneumothorax"
]):
    return "Chest/Lung"

# ILLNESS / SYSTEMIC
if any(k in x for k in [
    "illness", "covid", "flu", "overall body", "general"
]):
    return "Illness"

```

```

    return "Other"

injury_df["BodyPartGroup"] = injury_df["BodyPart_clean"].apply(map_body_part_group)

# Injury Detail
injury_df["InjuryDetail"] = injury_df["REASON"].str.extract(r';\s*(.*)', expand=False)

#Season Phase variable
def season_phase(m):
    if m in [10, 11]:
        return "Early"
    elif m in [12, 1]:
        return "Mid"
    else:
        return "Late"
injury_df["SeasonPhase"] = injury_df["Month"].apply(season_phase)

#Count of injuries by player/season
injury_df = injury_df.sort_values(['PLAYER', 'Season', 'DATE'])

injury_df["InjuriesThisSeason"] = (
    injury_df.groupby(['PLAYER', 'Season']).cumcount()
)

injury_df.to_csv("visualization_injuries_2021_2024.csv", index=False)

# %% [markdown]
# For the ML Model dataset, I need to collapse down to injury event. I consider an injury apart of the same event if SAME
player, SAME BodyPart, Injury rows reasonably close in time, No prolonged return to play period in between them (>10
days)

```



```

# %%
#ML Model Dataset
event_df = injury_df.copy()

print(event_df.columns)
#Sort
event_df = event_df.sort_values(["PLAYER", "BodyPart", "InjuryDetail", "DATE"])

# Difference between consecutive injury reports
event_df["date_diff"] = event_df.groupby(
    ["PLAYER", "BodyPart"]
)["DATE"].diff().dt.days

#new injury if gap is > 10 days or first occurrence
event_df["new_injury_flag"] = event_df["date_diff"].isna() | (event_df["date_diff"] > 10)

#create an injury event id to track every unique case
event_df["injury_event_id"] = event_df.groupby(
    ["PLAYER", "BodyPart"]
)["new_injury_flag"].cumsum()

#Collapsing the data down to each incident
collapsed = event_df.groupby(
    ["PLAYER", "Season", "BodyPart", "BodyPart_clean", "BodyPartGroup", "injury_event_id"]
).agg({
    "DATE": ["min", "max", "count"],
    "SeasonPhase": "first",
    "InjuryDetail": "first",
    "REASON": "first",
    "TEAM": "first",
    "GAME": "first",
    "STATUS": "first"
})

```

```

collapsed.columns = [
    "InjuryStart", "InjuryEnd", "GamesMissed",
    "SeasonPhase", "InjuryDetail", "Reason",
    "Team", "Game", "Status"
]
collapsed = collapsed.reset_index()

#Duration of Injury
collapsed["DurationDays"] = (collapsed["InjuryEnd"] - collapsed["InjuryStart"]).dt.days

# Event-based injury count
collapsed = collapsed.sort_values(["PLAYER", "Season", "InjuryStart"])
collapsed["InjuriesThisSeason"] = collapsed.groupby(
    ["PLAYER", "Season"]
).cumcount()

collapsed.to_csv("ml_injury_events_2021_2024.csv", index=False)

# %% [markdown]
# Merging in the season stats data

# %%
s22 = pd.read_csv("2021_Stats.csv")
s23 = pd.read_csv("2022_Stats.csv")
s24 = pd.read_csv("2023_Stats.csv")

# %%
#assigning season
s22["Season"] = 2022
s23["Season"] = 2023
s24["Season"] = 2024
stats = pd.concat([s22, s23, s24], ignore_index=True)

#Flag players that were traded

```

```

stats["has_multi_team_total"] = stats["Team"].str.contains("TM", case=False, na=False)

final_rows = []

for (player, season), group in stats.groupby(["Player", "Season"]):
    # if player traded, use total row
    if group["has_multi_team_total"].any():
        row = group[group["has_multi_team_total"]].iloc[0]
        final_rows.append(row)

    # if a player reamish on same team
    else:
        row = group.iloc[0]
        final_rows.append(row)

stats_clean = pd.DataFrame(final_rows)

# Create player_key to match injury dataset
stats_clean["player_clean"] = stats_clean["Player"].str.strip()
stats_clean["player_key"] = (
    stats_clean["player_clean"]
    .str.lower()
    .str.replace("[^a-z]", "", regex=True)
    .str.replace("\s+", " ", regex=True)
    .str.strip()
)

#Create Avg Mins Per Game Variable
stats_clean["MPG"] = stats_clean["MP"] / stats_clean["G"]

#Create Avg Points per Game Variable
stats_clean["AVGPTS"] = stats_clean["PTS"] / stats_clean["G"]

#Create Avg FTS per Game Variable

```

```

stats_clean["AVGFTS"] = stats_clean["FT"]/stats_clean["G"]

#Create Avg 3PTS per Game Variable
stats_clean["AVG3PTS"] = stats_clean["3P"]/stats_clean["G"]

stats_clean.to_csv("stats_2021_2024.csv", index=False)

# %%
# Load datasets
ml_df = pd.read_csv("ml_injury_events_2021_2024.csv")
viz_df = pd.read_csv("visualization_injuries_2021_2024.csv")
stats_clean = pd.read_csv("stats_2021_2024.csv")

#Normalizing both names, have to remove speical characters
def normalize_for_merge(name):
    if pd.isna(name):
        return name

    # lowercase and remove spaces
    name = name.lower().strip()

    # remove accents from international name
    name = ".join(
        c for c in unicodedata.normalize('NFKD', name)
        if not unicodedata.combining(c)
    )

    # cmake sure apostrophephes the same
    name = name.replace("'", "")

    # remove periods entirely
    name = name.replace(".", "")

```

```

# keep letters, numbers, spaces, periods, hyphens, apostrophes
name = re.sub(r"^[a-z0-9\.\-\']+", "", name)

# collapse multiple spaces
name = re.sub(r"\s+", " ", name).strip()

return name

ml_df["merge_key"] = ml_df["PLAYER"].apply(normalize_for_merge)
viz_df["merge_key"] = viz_df["PLAYER"].apply(normalize_for_merge)
stats_clean["merge_key"] = stats_clean["Player"].apply(normalize_for_merge)

# merging ml injury data with stats
merged_ml_df = ml_df.merge(
    stats_clean,
    left_on=["merge_key", "Season"],
    right_on=["merge_key", "Season"],
    how="left",
    indicator=True
)

# merging viz injury data with stats
merged_viz_df = viz_df.merge(
    stats_clean,
    left_on=["merge_key", "Season"],
    right_on=["merge_key", "Season"],
    how="left",
    indicator=True
)

# Extract missing rows
missing = merged_ml_df[merged_ml_df["_merge"] == "left_only"]

```

```

# Show first 20 unmatched player names to find possible issues
missing_players = missing["PLAYER"].unique()
missing_players[:20]

# Extract missing rows
missing = merged_viz_df[merged_viz_df["_merge"] == "left_only"]

# Show first 20 unmatched player names
missing_players = missing["PLAYER"].unique()
missing_players[:20]

#drop unmatched rows
ml_final = merged_ml_df[merged_ml_df["_merge"] == "both"].copy()
viz_final = merged_viz_df[merged_viz_df["_merge"] == "both"].copy()

#Create a lag variable for ml dataset

#Here I am making a lag variable
history_df = ml_final[["PLAYER", "Season", "InjuriesThisSeason", "Rk"]].drop_duplicates()
history_df["Next_Season"] = history_df["Season"] + 1
history_df = history_df.rename(columns={
    'InjuriesThisSeason': 'Injuries_Prev_Season',
    'Rk': 'Rank_Prev_Season'
})
ml_final = ml_final.merge(
    history_df[["PLAYER", "Next_Season", "Injuries_Prev_Season", "Rank_Prev_Season"]],
    left_on=["PLAYER", "Season"],
    right_on=["PLAYER", "Next_Season"],
    how='left'
)

ml_final["Injuries_Prev_Season"] = ml_final["Injuries_Prev_Season"].fillna(0)
ml_final["Rank_Prev_Season"] = ml_final["Rank_Prev_Season"].fillna(-1)

```

```

ml_final.drop(columns=['Next_Season'], inplace=True)

# %%
#Missing Value problem
print(len(ml_final))
ml_final.isna().sum().sort_values(ascending=False)
#Awards is main source of problem
ml_final = ml_final.drop(columns = ["Awards", "BodyPart_clean"])
ml_final = ml_final.dropna()
print(len(ml_final))

# %%
ml_final.to_csv("final_ml_dataset_with_stats.csv", index=False)
viz_final.to_csv("final_viz_dataset_with_stats.csv", index=False)

# %% [markdown]
# Machine Learning
# - Unsupervised Learning K- means clustering, goal is to find player archtypes for injury

# %%
# need to create a new data frame for this, where we take only the last record of every player each season
# goal is to not have a row for every injury event, but one row for each of these injured players during a season
df_profiles = ml_final.sort_values(['PLAYER', 'Season', 'InjuriesThisSeason']).drop_duplicates(subset=['PLAYER', 'Season'],
keep='last').copy()

#select features for clustering
features = ['Age', 'MPG', 'InjuriesThisSeason', 'AVGPTS', 'AVGFTS', 'AVG3PTS', 'Pos']
#assign features and drop any nas that can cause errors
X_cluster = df_profiles[features].dropna()
num_features = ['Age', 'MPG', 'InjuriesThisSeason', 'AVGPTS', 'AVGFTS', 'AVG3PTS']
cat_features = ['Pos']

preprocessor_cluster = ColumnTransformer(
    transformers=[

```

```

    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
])

#Optimize the kmeans by finding optimal k through SSD

ssd_vec = []
cluster_range = range(1,20)
for k in cluster_range:
    #kmeans model, finding the optimal k
    kmeans_pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor_cluster),
        ('cluster', KMeans(n_clusters=k, random_state=42, n_init=10))
    ])
    kmeans_pipeline.fit(X_cluster)

    ssd = kmeans_pipeline.named_steps['cluster'].inertia_
    ssd_vec.append(ssd)

# Plotting to find knee point
plt.figure(figsize=(10, 6))
plt.plot(cluster_range, ssd_vec, marker='o', linestyle='-', color='b')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Sum of Squared Distances (Inertia)')
plt.xticks(cluster_range)
plt.grid(True, alpha=0.3)

# %%
optimal_k = 4
final_kmeans_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor_cluster),
    ('cluster', KMeans(n_clusters=optimal_k, random_state=42, n_init=10))
])

```



```

])

# Fit to the data
final_kmeans_pipeline.fit(X_cluster)

#create a results dataframe
X_cluster['Cluster'] = final_kmeans_pipeline.named_steps['cluster'].labels_

# %%
#Results from Kmeans
print("Size of each cluster")
print(X_cluster['Cluster'].value_counts().sort_index())
print("\n")

print("Cluster Average Numeric Variables")
print(X_cluster.groupby('Cluster')[num_features].mean())
print("\n")

print("Distribution of each position in each cluster")
print(X_cluster.groupby('Cluster')['Pos'].value_counts(normalize=True).unstack().fillna(0))

# %%
# add data to graph
clusters = ['Durable Rotation', 'Elite Stars', 'Bench/Rookies', 'Fragile Veterans']
mpg = [28.3, 34.3, 14.1, 19.8]
injuries = [1.13, 2.45, 0.79, 1.95]

# Basic Scatter Plot
plt.scatter(mpg, injuries, color='blue')

# label each cluster
for i, label in enumerate(clusters):
    plt.text(mpg[i], injuries[i], label)

```

```

# Label
plt.title('Archetype Scatter: Workload versus Injury Risk')
plt.xlabel('Avg Minutes Per Game')
plt.ylabel('Avg Injuries Per Season')
plt.grid(True, alpha = 0.5)

plt.show()

# %% [markdown]
# Machine Learning
# - Supervised Learning (Predict Games Missed based on variables) Ex. Injury Type, Body Part, Position
# - 3 categories, low (1-2 games missed), medium (3-7 games missed), high (8 or more games missed)

# %%
df = ml_final.copy()

# Create Target and Features
def severity_label(x):
    if x <= 2:
        return "Low"
    elif x <= 7:
        return "Medium"
    else:
        return "High"

df['Severity'] = df['GamesMissed'].apply(severity_label)

#Assign Features
features = ['InjuriesThisSeason', 'Age', 'Pos', 'MPG', "BodyPart", 'Team_y', 'SeasonPhase', "FG%", 'FT%', 'AVGPPTS',
'AVGFTS', 'AVG3PTS', 'Injuries_Prev_Season', 'Rank_Prev_Season']
X = df[features]
y = df['Severity']

#Create a train test split

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)

#Pre Processing
cat_features = ['Pos', 'BodyPart', 'Team_y', 'SeasonPhase']
num_features = ['InjuriesThisSeason', 'Age', 'MPG', 'FG%', 'AVGPTS', 'AVGFTS', 'AVG3PTS', 'Injuries_Prev_Season',
'Rank_Prev_Season']

preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
])

# %%

#Logistic Regression
log_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(multi_class='multinomial', max_iter=1000, random_state=42))
])

#Random Forest Model
#Preprocessing variables so it encodes categoricals and keeps numerical
rf_model = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("rf", RandomForestClassifier(
        n_estimators=500,
        max_depth=None,
        min_samples_split=5,
        min_samples_leaf=2,
        random_state=42
    ))
])

# %%

print("Logistic Regression Model Results")

```

```

log_model.fit(X_train, y_train)
y_pred = log_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

print("Random Forest Model Results")
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

# %%
# Data
metrics = ['Accuracy', 'High Severity Precision']
log_scores = [0.61, 0.46]
rf_scores = [0.73, 0.86]

x = np.arange(len(metrics))
width = 0.35

# Basic Plot bar plot
plt.bar(x - width/2, log_scores, width, label='Logistic Regression')
plt.bar(x + width/2, rf_scores, width, label='Random Forest')

# Label axis and bars
plt.xticks(x, metrics)
plt.ylabel('Score')
plt.title('Model Comparison')
plt.legend()

```

```
plt.show()
```